

Programmazione Orientata agli Oggetti
Appello del 24/9/2025

Parte 1: Quiz

Rispondere alle seguenti 8 domande cerchiando la lettera delle/a risposte/a appropriate/a su questo foglio. **Scrivere nome, cognome e matricola su OGNI FOGLIO** che si consegna.

Cosa rappresenta il principio Single Responsibility (SOLID)?

- a) Una classe deve avere una sola sottoclasse
- b) Una classe deve avere un solo costruttore
- c) Una classe deve avere una sola ragione per cambiare
- d) Una classe deve implementare una sola interfaccia

Quale delle seguenti affermazioni sull'ereditarietà multipla in Java è vera?

- a) È supportata completamente per le classi
- b) Non è supportata né per classi né per interfacce
- c) È supportata solo attraverso le interfacce
- d) È supportata solo per classi astratte

Cosa succede quando si tenta di accedere a un elemento oltre i limiti di un array?

- a) Viene restituito null
- b) Viene restituito 0 o false
- c) Viene lanciata una `ArrayIndexOutOfBoundsException`
- d) L'array si espande automaticamente

Quale delle seguenti affermazioni sui metodi statici è corretta?

- a) Possono accedere direttamente alle variabili di istanza
- b) Possono essere sovrascritti (override) nelle sottoclassi
- c) Appartengono alla classe, non all'istanza
- d) Devono sempre essere pubblici

Quale delle seguenti affermazioni sul garbage collector di Java è vera?

- a) Deve essere chiamato esplicitamente dal programmatore
- b) Libera automaticamente la memoria degli oggetti non referenziati
- c) Funziona solo per gli oggetti creati con `new`
- d) Non può mai essere eseguito durante l'esecuzione del programma

Nel pattern Factory Method, quale è il vantaggio principale?

- a) Riduce il numero di interfacce nel sistema
- b) Delega la creazione di oggetti alle sottoclassi
- c) Elimina la necessità di utilizzare costruttori
- d) Garantisce che tutti gli oggetti siano singleton

Quale delle seguenti affermazioni sulle eccezioni checked in Java è vera?

- a) Devono essere gestite obbligatoriamente con try-catch o throws
- b) Vengono lanciate solo a runtime
- c) Non possono essere ereditate
- d) Sono sempre figlie di RuntimeException

Nel pattern Decorator, quale è lo scopo principale?

- a) Creare una famiglia di algoritmi intercambiabili
- b) Aggiungere funzionalità a un oggetto dinamicamente senza alterarne la struttura
- c) Garantire che ci sia una sola istanza di una classe
- d) Separare la costruzione di un oggetto dalla sua rappresentazione

Parte 2: Sviluppo di un programma Java

Considerare la situazione descritta sotto e implementare tipi e metodi in Java per supportare le funzionalità richieste. È importante che si specifichino adeguatamente tipi di dati astratti e contratti e che si ottemperi al principio di singola responsabilità. L'adozione di "clean coding" e di pattern di design ove appropriato è pure importante. Escludere ogni problematica relativa alla interfaccia utente e alla persistenza. Procedere top-down.

Scrivere nome, cognome e matricola su OGNI FOGLIO che si consegna.

PizzaManager è un sistema di gestione per una pizzeria moderna che offre servizio al tavolo sia all'interno che all'aperto. La pizzeria "Ai Rizzi" dispone di 25 tavoli distribuiti in due aree: 15 tavoli interni (da 2 a 8 coperti ciascuno) e 10 tavoli esterni (da 2 a 6 coperti ciascuno). I tavoli possono essere uniti per ospitare gruppi più numerosi, creando configurazioni personalizzate.

Il personale di sala è composto da 6 camerieri, ognuno responsabile di una specifica area di tavoli. Ogni cameriere ha un codice identificativo, nome, cognome.

Il menu è organizzato in categorie: pizze, antipasti, primi piatti, secondi di carne e pesce, contorni, dolci della casa e bevande. Ogni pietanza ha un codice, nome, descrizione, prezzo, categoria, e disponibilità (alcune specialità sono disponibili solo in certi giorni).

I clienti si siedono ai tavoli e comunicano gli ordini al cameriere di zona. Ogni ordine è caratterizzato da timestamp, numero tavolo (o gruppo di tavoli), cameriere responsabile, lista delle pietanze con quantità, stato dell'ordine (in preparazione, servito, pagato), modalità di pagamento (contanti, carta, buoni pasto) e importo totale. Gli ordini possono avvenire in più fasi durante il servizio - ad esempio prima le bevande e i primi, poi le portate principali. Durante una serata, allo stesso tavolo si possono susseguire diversi gruppi di clienti. Il sistema deve tracciare separatamente ogni servizio, permettendo di distinguere gli ordini di clienti diversi anche se utilizzano lo stesso tavolo fisico. Il sistema deve gestire in memoria (senza persistenza o interfaccia utente) quanto serve per queste operazioni principali:

Visualizzare gli ordini attivi in tempo reale, mostrando per ogni ordine non ancora pagato: tavolo/i coinvolti, cameriere responsabile, lista dettagliata delle pietanze ordinate con quantità e prezzi, stato di preparazione, orario dell'ordine e importo parziale.

Calcolare il conto di un tavolo in un momento specifico, sommando tutte le pietanze ordinate e non ancora pagate, generando il totale da pagare con il dettaglio completo.

Generare il resoconto serale con il totale degli incassi ripartito per cameriere, includendo numero di tavoli serviti, numero totale di coperti, incasso lordo per cameriere, ripartizione degli incassi per categoria di menu (pizze, antipasti, primi, etc.) e modalità di pagamento utilizzate.

Implementare le classi Java necessarie con i relativi metodi, gestendo appropriatamente le relazioni tra tavoli, camerieri, ordini e pietanze, e fornendo i metodi per le operazioni richieste.

È obbligatorio produrre anche il codice del Main che illustri esempi di chiamata dei metodi che implementano queste operazioni.

